Intro

안녕하세요, 백엔드 개발자 최두용 입니다.

1996. 10. 03
chlendyd7@gmail.com
010-4632-6222
https://github.com/chlendyd7

Summary

안정적이고 확장성 있는 백엔드 구조를 지향하는 개발자입니다.
Java, Python 기반 대용량 처리, 비동기 파이프라인, 동시성 제어, 실시간 이벤트 처리 구조를 개발했습니다.
초당 5,000+의 데이터의 환경에서 시스템 운영 및 ERP/설비 서버/품질 서버 등 시스템 간 통신을 담당하였습니다
최근에는 단순 LLM 연동을 넘어 Agentic AI, MCP, LangGraph 관점의 실행 구조까지 확장하며 서비스형 AI 아키텍처를 직접 설계·구현하고 있습니다.

핵심 역량

성능 최적화

- N+1 문제 해결, Redis 캐시/비동기 파이프라인, 병목 구간 개선
- 장애를 줄이는 구조 설계와 운영 안정성 확보에 집중

안정적인 백엔드 설계

- 동시성 이슈, 분산 환경 스케줄 제어, 원자성 보장, 운영 자동화 경험
- 서비스 상황에 따라 단순 구현보다 안전한 처리 구조 우선 설계

업무 시스템 연계 및 데이터 인터페이스 설계

- ERP, MES, PLC, WMS 등 레거시/현장 시스템과의 데이터 연계 경험
- 정합성과 운영 제약을 고려한 인터페이스 및 배치 구조 설계

실시간/이벤트 기반 시스템 경험

- REST, SSE, WebSocket, MQTT, WebRTC 기반 데이터 흐름 구현
- 사용자 응답성과 이벤트 처리 특성을 고려한 구조 설계 경험

Agentic AI 및 서비스형 AI 아키텍처

- Tool-calling, MCP, LangGraph 관점의 Agentic AI 실행 구조 설계
- 단순 모델 연동을 넘어 생성·검증·연동·운영까지 포함한 AI 서비스 구조 확장

주요 기술

Backend
- Spring Boot, Django, FastAPI

Data / Messaging
- PostgreSQL, Redis, DynamoDB, Kafka, EMQX

Infra / Ops
- Docker, Kubernetes, Jenkins, Sentry, Jira, k6

AI / Agentic
- LangChain, LangGraph, MCP, RAG, VectorDB

경력

현대 위아 2026.04 ~ ing

시스템 유지보수 및 PM

- 열관리 시스템 MES 유지보수 (Spring, Vue)
- 사내 AI Agentic 스튜디오 개발 (Spring, FastAPI, React)
- SCADA 시스템 PM (CIMON)
- 프로젝트 생애주기 서비스 개발

애기야가자 2023. 11. 20 ~ 2024. 08. 15

서버 및 백엔드 개발

- 200MAU 서버 및 API 운영
- N+1, 캐싱, 비동기 처리로 성능 30 ~ 80% 개선

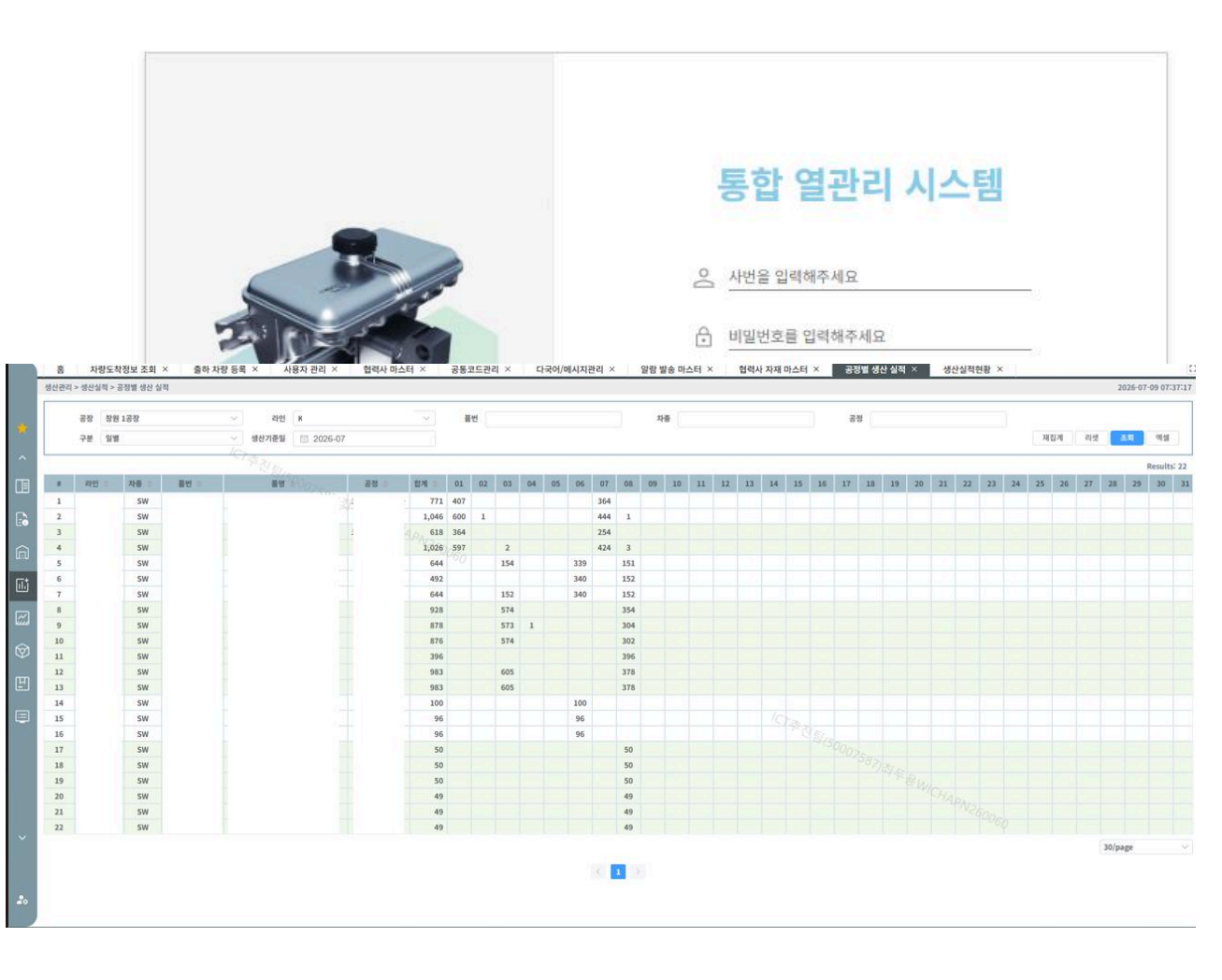
TMS MES

TMS MES 2026. 04 ~

열관리 MES 프로그램 유지보수

Spring Boot , Vue3
PostgresSQL, MyBatis

풀스택: 최두용, 홍현우 차장님
PM: 최두용, 오창희 책임님



프로젝트 내용

- 제조 현장에서 발생하는 데이터를 ERP, 각종 레거시 프로그램을 연동하는 MES 프로젝트
- 초당 50,000+ 데이터를 DB to DB 및 MES와 연동
- 데이터 정합성과 보안성이 중요한 운영 시스템으로, 기능 개발뿐 아니라 운영 안정성과 사용성을 높이기 위한 유지보수, UI/UX 개선, 보안 구조 개선을 함께 수행

본인의 역할

- 풀스택 앱 유지보수
- 정보보안팀과 시큐어 코딩 협업
- PM 역할로 현업과 소통 및 협력사 개발자 일정 및 환경 조성

주요 성과

유지보수

- 50+의 프론트 및 백엔드 SM, 기능 개선을 담당
- 테이블 생성 및 구조 변경, 타 시스템과 연동
- 프로시저와 배치 처리를 통해 데이터 자동 집계 및 정합성 유지
- SQL 하드코딩 대신 Java 함수로 로직을 분리해 재사용성과 유지보수성 향상

PM

- 현업과 소통하며 요구사항을 정리하고 개발 항목으로 구체화
- 협력사 개발자와 일정, 우선순위, 작업 범위를 조율하며 개발 진행 지원
- 생산성을 높히기 위해 개발 및 인터페이스 문서 등을 점검·조성
- 운영 이슈 발생 시 원인 파악과 대응 방향을 정리하며 현업–개발 간 브릿지 역할 수행

UI/UX 개선

- 현업 요구사항을 단순 반영하는 데 그치지 않고, 실제 사용 흐름을 분석해 더 나은 UI/UX 대안을 제안하고 구현
- 유사 항목 집계 화면에 전체 항목 컬럼을 추가해 세부 항목과 총합을 한눈에 비교할 수 있도록 개선
- 개인정보 비식별화 기능은 별도 토글 방식 대신 클릭 즉시 비식별화되는 방식으로 변경 화면 복잡도를 줄이고 사용성을 향상

보안 취약점 대응

- UI 권한 체크에 머물러 있던 기존 구조를 개선해, API 레벨에서 인증·인가를 검증하는 방식으로 보안 구조 강화
- 사용자별 목록 비교 수준의 권한 관리에서 나아가, URL 레벨 권한 체크 기반으로 테이블 및 API 구조를 재설계
- 개인정보 보호를 위해 마스킹 정책을 수립하고 실제 서비스에 적용하여, 운영 환경에서의 보안성과 준수 수준 강화

애기야가자

애기야가자 2023. 11 ~ 2024. 08
사용자 증가로 인한 조회 병목과 비동기 처리 지연을 개선한 서비스형 백엔드 프로젝트

DRF / AWS / Redis / Celery / PostgresSQL
DynamoDB / FireBase / Sentry / Notion

BE: 최두용, 시니어 개발자
FE: 3인

애기야가자
서비스 소개
회사 소식
애기야가자 채용
사장님 페이지
CHILDREN
RUN THE WORLD
아이들이 자유롭게 뛰어다니는 세상을 꿈꿉니다.
앱 다운로드

본인의 역할

- 백엔드 서비스 전반 개발 및 API 성능 개선
- ORM N+1 문제 분석 및 캐시/DB 구조 개선
- 비동기 처리 및 실시간 상태 전달 구조 구현
- API 문서화 및 협업 인터페이스 정비

주요 성과

조회 병목 개선

ORM N+1 쿼리 해결
반복 조회 데이터 Redis 캐시로 분리, DynamoDB를 이용해 데이터 이관

- API 응답 시간: 800ms → 300ms
- 캐시 대상 조회: 129ms → 24ms

후처리 비동기화

알림, API 연동, 후처리 작업 파이프라인으로 인해 api 지연 및 서버 부하 발생
Celery + Redis 기반 비동기 워커 구조 도입, Retry/Health Check를 추가해 안정성 증가

- 처리 가능 작업량: 1,000건 → 5,000건

실시간 상태 전달 개선

실시간 알림/상태 전달이 필요했지만 WebSocket 우선 도입은 운영 복잡도 증가로 보였습니다
단방향 이벤트 전송이 필요한 영역이 SSE를 적용해 실시간성을 유지하며 구조를 단순화했습니다

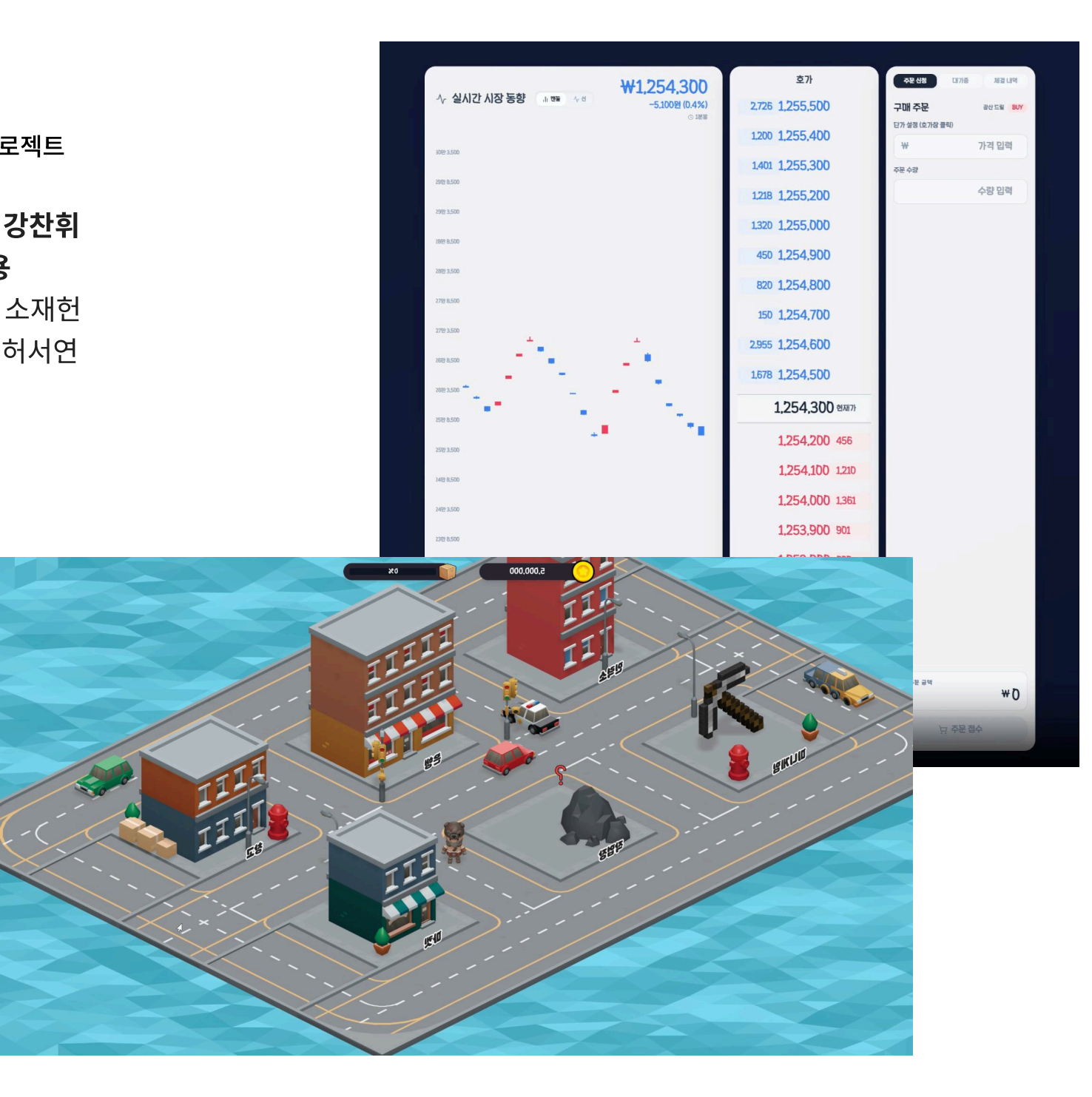
- 운영 복잡도 및 소켓 유지 비용 감소

Click-CEO

Click-CEO 2026. 02 ~ 2026. 04
실시간 주문 집계 및 호가창, 모니터링 시스템의 성능과 안정성을 개선한 프로젝트

Spring Boot / Kafka / Redis
Jenkins / K8s / Juint / K6
Jira / Webhook

BE: 최두용 / 강찬휘
Infra: 최두용
DE: 김은수 / 소재헌
FE: 손서영 / 허서연



본인의 역할

- 주문 집계 API 성능 개선 및 Redis Pipeline 도입
- Polling 기반 구조를 WebSocket 기반 실시간 구조로 전환
- 멀티 스레드/분산 락/K8s 환경 제어를 통해 안정성 개선
- Jenkins CasC, K8s 배포 및 테스트 자동화

주요 성과

연결 실패율 44% → 0%

기존 3초마다 풀링하는 구조를 WebSocket으로 바꾸자 Stress test에서 44% 연결 실패율을 보였습니다

원인

1. 봇 주문 생성 로직이 단일 스레드로 병목되며 WebSocket 연결 리소스가 부족했습니다
2. 봇이 1초 단위로 대량의 주문을 생성하고 있었습니다

해결

1. 봇 주문을 독립적 멀티 스레드로 분리하였습니다(연결 실패율 해결)
2. 봇 주문을 1초가 아닌 Stream으로 Redis에 흘려 부하를 분산하였습니다
3. Kafka를 사용해 메세지 브로커로 만들어 서비스별 구독 및 배치 처리로 부하를 분산하였습니다

무중단 배포 구현

실시간 시뮬레이션을 목표했기에 업데이트가 되더라도 중단 없이 시뮬레이션이 계속 흘렀으면 했습니다
Jenkins와 k8s를 사용하여 CI/CD 시 build와 pod를 갈아끼우며 무중단 배포를 구현하였습니다
좋은 로직을 위해 Jenkins를 CasC 기반 배포와 k6, junit 테스트를 추가하였습니다

멀티 Pod 환경 스케줄 제어

K8s replicas: 2 설정으로 Pod2개를 운영 중

원인

1. 배치 작업이 각 Pod에서 동시 실행되고 주문 정산 데이터 중복처리와 봇 주문 중복 현상이 발생했습니다

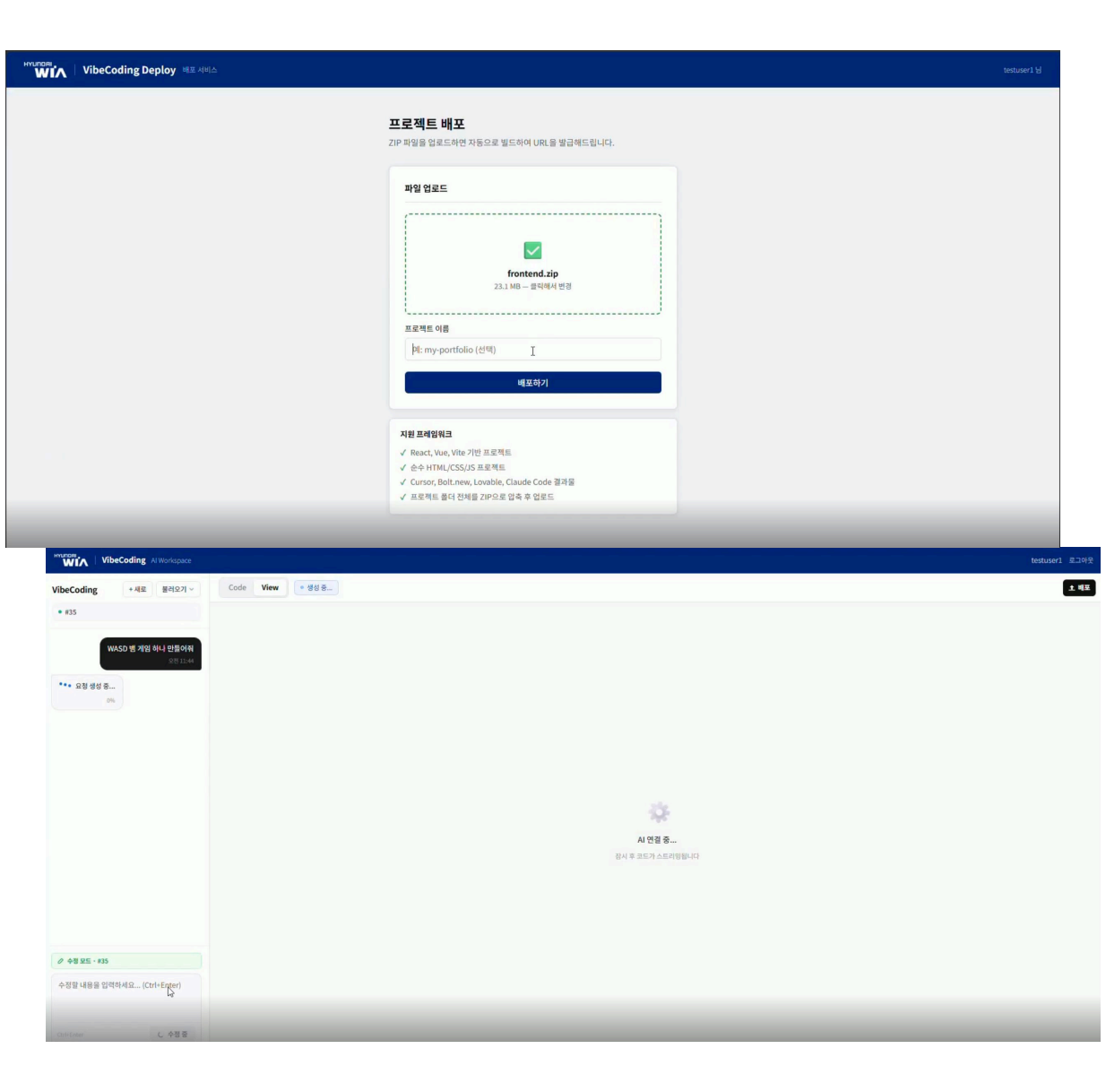
해결

1. MSA로 서비스 분리하기에는 일정 제약으로 인해 구조적 분리보다 스케줄 제어 우선 적용
2. ShedLock을 통해 K8s 멀티 Pod 환경의 중복 실행 문제 해결

AI Agent Studio

AI Agent Studio 2026. 05 ~
사내 업무용 웹앱의 생성·연동·검증·배포를 표준화하는 Agentic AI 플랫폼

Spring Boot , FastAPI, React
Claude API, Tool Calling, Agent Loop
MCP, LangChain, LangGraph, Multi-Agent



배경

- 생성형 AI 확산과 AX 교육 이후, 현업이 직접 바이브코딩 기반 업무용 웹앱을 만드는 수요가 증가했지만 이를 안전하게 수용할 표준 플랫폼이 부재
- 개별적으로 생성된 앱은 보안 통제 밖에서 운영되고, ERP/MES 등 사내 시스템과 연결되지 못하며, 배포·승인·운영 체계 또한 없어 조직 자산으로 축적되기 어려움
- 이를 해결하기 위해 앱 생성부터 사내 시스템 연동, 보안 검증, 내부 배포까지 표준화하는 사내 Agentic AI 플랫폼 PoC 기획·구현

주요 성과

Agentic Loop 직접 구현

- 단순 1회성 LLM 호출이 아닌, 오픈소스 분석을 통한 tool-calling 기반 Agentic Loop를 직접 설계·구현
- 앱 생성 과정에서 필요한 정보를 조회하고 결과를 반영하는 자기검증형 실행 구조를 구현
- Multi-Agent를 도입 해 각 단계별 적합한 모델 배치, 토큰 절약
- LangGraph 기반 상태 오케스트레이션으로 유지보수 및 확장 가능하도록 변경 예정

MCP 기반 사내 시스템 연동 표준화

- ERP, MES 등 레거시 시스템과의 연결을 위해 MCP 기반 표준 연동 계층 설계
- 현업 사용자가 API 구조를 직접 알지 않아도, 허용된 데이터와 기능만 선택해 사용할 수 있도록 구성
- tool-calling과 MCP를 분리된 기능이 아닌 공통 도구 인프라 관점에서 설계

보안·배포·운영 통제 구조 설계

- 앱 생성 이후 자동 보안 점검 → 승인 → 격리 배포로 이어지는 운영 구조 정의
- Secret 탐지, 외부 URL 호출 차단, 인증/인가 점검, 감사 로그 등 내부 플랫폼에 필요한 통제 요소 반영
- Docker 기반 프로젝트 격리 배포와 리소스 관리 방향 수립

플랫폼 수준 MVP 기획 및 확장 방향 수립

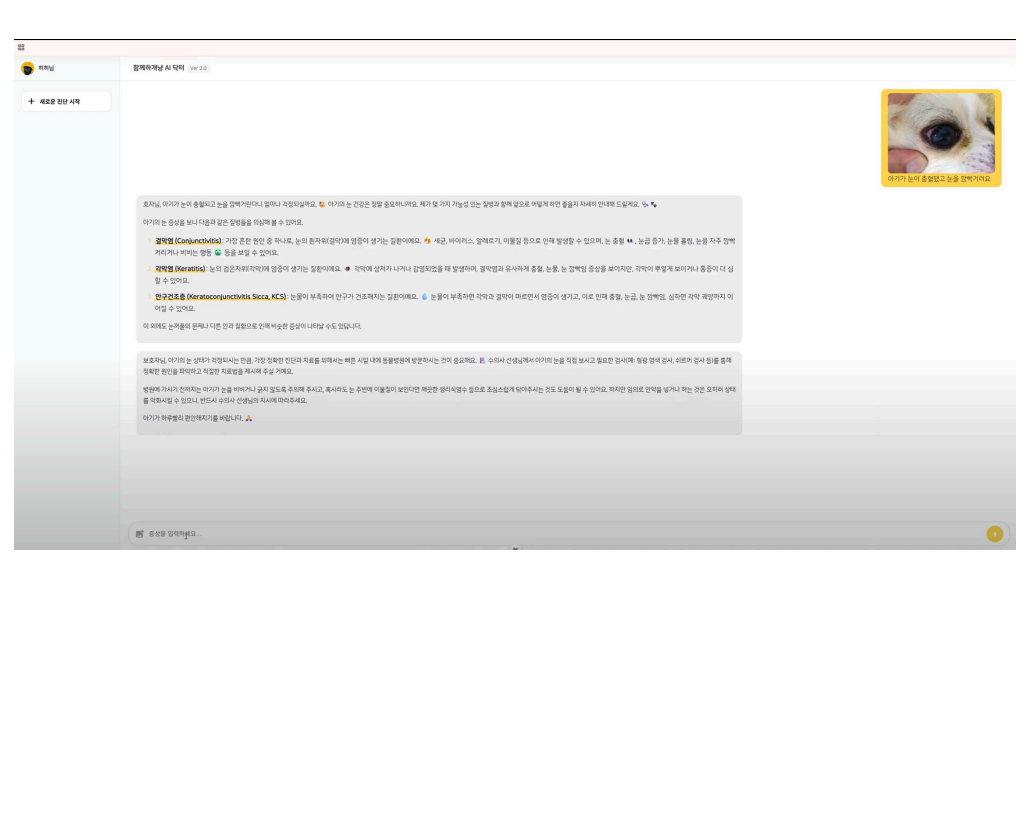
- 3개월 내 MVP 구축, ICT 추진팀 파일럿을 목표로 단계별 로드맵과 KPI 수립
- 단순 코드 생성 도구가 아닌, 생성·연동·배포·운영을 포함한 내부 AI 앱 플랫폼으로 방향 정의
- 향후 앱 카탈로그, 운영 대시보드 등으로 확장 가능한 구조 설계

함께하개냥

함께하개냥 2025. 10 ~ 2025. 12
멀티모달 AI 기반 실시간 진단 서비스의 스트리밍 응답과 미디어 처리를 최적화한 프로젝트

Django, FastAPI, Vue, PostgreSQL, Docker, Redis
embeding Vector

BE: 최두용
FE: 최두용, 하서영
UI/UX: 하서영



본인의 역할

- 전체 시스템 아키텍처 설계 및 API 구현
- FastAPI 기반 LLM 스트리밍 응답 구현
- 이미지 전처리 및 비동기 업로드 파이프라인
- Vector Embedding 기반 의미 검색 기능
- Redis 부하 최적화 및 성능 개선

주요 성과

LLM 스트리밍 응답 최적화

LLM 응답이 완성될 때까지 기다리면 사용자 대기 시간이 길어집니다.
FastAPI의 Stream과 AsyncGenerator를 활용해 응답을 청크 단위로 전송했습니다.
초기 응답 시간(TTFB)을 단축해 사용자 체감 성능을 개선했습니다.

- TTFB: 20초+ → 5초로 개선

이미지 전처리 및 업로드 최적화

Full HD 원본 이미지를 그대로 처리하여 AI 분석 비용 증가
전처리 파이프라인을 구성해 AI분석 데이터와 실제 데이터를 나누었습니다.
분석 데이터의 경우 해상도 축소, 포맷 변환을 진행, 경량화된 이미지만 AI에 전달해 처리 시간 및 비용을 단축했습니다.

- 전처리 시간: 15초 → 3초로 개선
- 전송 데이터 크기 대폭 감소

Vector Embedding 기반 의미 검색

과거 진단 결과와 현재 질문의 의미 유사도를 파악하기 위해
Vector Embedding을 활용한 의미 기반 검색을 구현했습니다.

- 의미 중심의 검색 결과 제공
- 사용자 질의에 맞는 관련 진단 결과 추천

결제 API 파이프라인 구현

사용자의 결제 요청이 들어오면, 결제 처리, DB 반영, 후처리 작업(알림, 정산)이 동시에 일어나면서 정합성 및 성능 문제 발생.
결제 트랜잭션 및 비동기 파이프라인 구현을 통해 안정성 증대

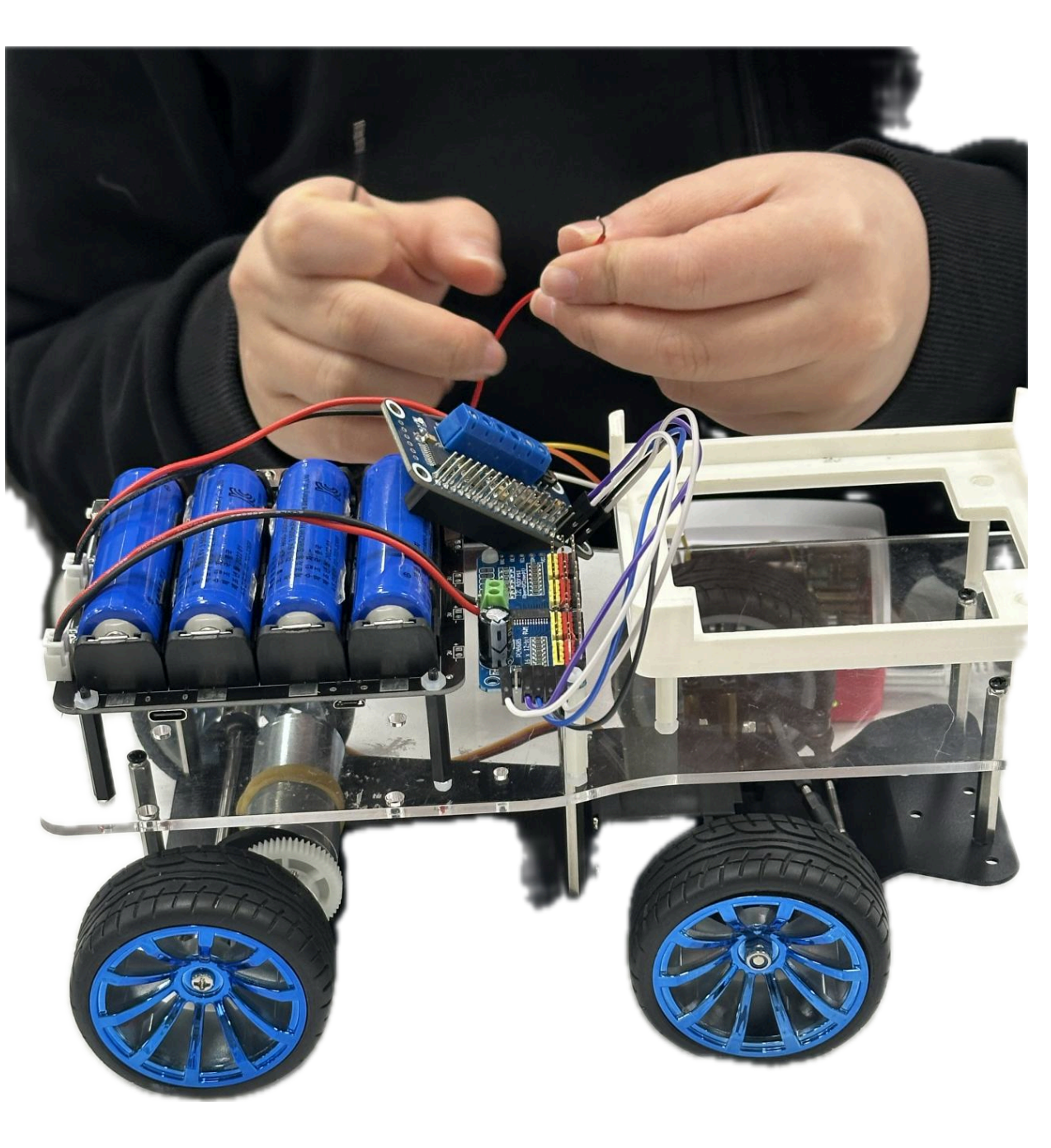
- 결제 정합성: 완벽 보장
- DB 무결성: 90% 이상 확보
- 비동기 후처리 안정화

Cescar

Cescar 2026. 01 ~ 2026. 02
IoT 디바이스와 웹 서비스를 연동해 실시간 영상 스트리밍과 이벤트 처리를 구현한 AIoT 프로젝트

Spring, FastAPI
MQTT(EMQX), WebRTC, YOLO, Ros2
Docker, PostgreSQL, Jira

BE: 최두용, 김건남, 김태영
IoT: 최두용, 김건남
AI: 정성윤
FE: 김승수



본인의 역할

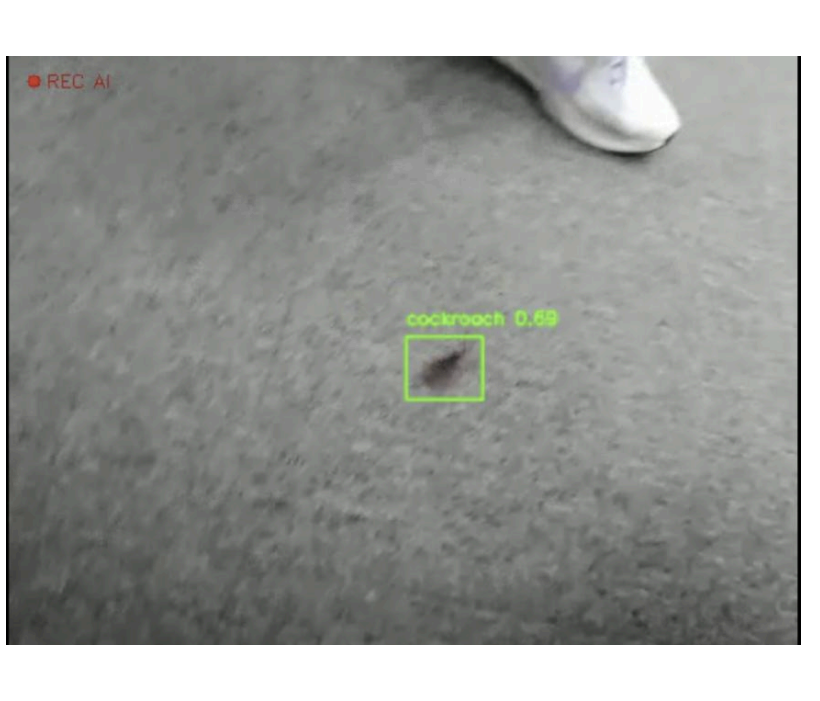
- MQTT 기반 IoT 이벤트 연동 및 상태 전달
- WebRTC signaling 및 실시간 영상 스트리밍 구조 구현
- Edge AI 감지 로직 및 비동기 업로드 처리
- Virtual Thread 기반 I/O 병목 개선

주요 성과

MQTT 기반 디바이스 연동

EMQX 기반 MQTT 브로커를 통해 IoT 디바이스의 이벤트와 상태를 수집하고
Pub/Sub 구조로 서버와 비동기 연동했습니다.

- 디바이스 이벤트 수집 구조 구현
- 경량 메시징 기반 실시간 상태 전달



Virtual Thread로 I/O 병목 개선

IoT 센서 데이터 처리와 DB/네트워크 I/O가 Thread Pool을 점유하면서 응답 지연이 발생했습니다.
Spring Virtual Thread를 도입해 I/O 작업을 경량화하고, blocking 구간을 분리해 처리 효율을 높였습니다.

- TPS 2배 이상 향상
- 6개 카메라 기준 TPS 안정성 확보

Edge AI 비동기 처리 및 FPS 최적화

YOLO 기반 감지, FFmpeg 인코딩, GCS 업로드가 한 흐름에서 동작하면서 FPS 저하가 발생했습니다.
Daemon Thread 분리와 하드웨어 가속, fallback 로직을 적용해
실시간 감지 루프와 업로드 작업을 분리했습니다.

- 실시간 FPS 저하 완화
- 무중단 클라우드 업로드 파이프라인 구성

FCM Web Push 및 GCS 업로드

감지 이벤트 발생 시 FCM 토큰을 활용한 Web Push 알림을 전송하고,
영상은 GCS Signed URL 기반으로 업로드해 클라이언트와 서버 간 전송 부담을 줄였습니다.

- 이벤트 기반 알림 구조 구현
- 대용량 미디어 업로드 경량화